

Dynamic Programming – Optimization Problems

0/1 Knapsack, Travelling Salesperson Problem (DP version), Optimal Binary Search Tree

GUIDE: Dr. Sriramya

Name: Jaasiel J S

1. 0/1 Knapsack Problem (Dynamic Programming)

<pre>main.py 1 def knapsack(weights, values, capacity): 2 n = len(weights) 3 dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)] 4 for i in range(1, n + 1): 5 for w in range(capacity + 1): 6 if weights[i - 1] <= w: 7 dp[i][w] = max(8 values[i - 1] + dp[i - 1][w - weights[i - 1]], 9 dp[i - 1][w] 10) 11 else: 12 dp[i][w] = dp[i - 1][w] 13 return dp[n][capacity] 14 15 weights = [1, 3, 4, 5] 16 values = [1, 4, 5, 7] 17 capacity = 7 18 print("Maximum Profit:", knapsack(weights, values, capacity))</pre>	Run Output Maximum Profit: 9 --- Code Execution Successful ---
---	---

2. Travelling Salesperson Problem (Dynamic Programming)

<pre>main.py 1 from functools import lru_cache 2 3 def tsp(graph): 4 n = len(graph) 5 6 @lru_cache(None) 7 def visit(mask, pos): 8 if mask == (1 << n) - 1: 9 return graph[pos][0] 10 ans = float('inf') 11 for city in range(n): 12 if mask & (1 << city) == 0: 13 ans = min(ans, graph[pos][city] + visit(mask (1 << city), 14 city)) 15 return ans 16 17 return visit(1, 0) 18 19 graph = [20 [0, 10, 15, 20], 21 [10, 0, 35, 25], 22 [15, 35, 0, 30], 23 [20, 25, 30, 0] 24] 25 print("Minimum Cost:", tsp(graph))</pre>	Run Output Minimum Cost: 80 --- Code Execution Successful ---
---	--

3. Optimal Binary Search Tree (OBST)

<pre>main.py 1 def optimal_bst(keys, freq): 2 n = len(keys) 3 cost = [[0 for _ in range(n)] for _ in range(n)] 4 5 for i in range(n): 6 cost[i][i] = freq[i] 7 8 for length in range(2, n + 1): 9 for i in range(n - length + 1): 10 j = i + length - 1 11 cost[i][j] = float('inf') 12 total_freq = sum(freq[i:j + 1]) 13 for r in range(i, j + 1): 14 left = cost[i][r - 1] if r > i else 0 15 right = cost[r + 1][j] if r < j else 0 16 cost[i][j] = min(cost[i][j], left + right + total_freq) 17 18 return cost[0][n - 1] 19 20 keys = [10, 20, 30] 21 freq = [34, 8, 50] 22 print("Optimal BST Cost:", optimal_bst(keys, freq))</pre>	Output Optimal BST Cost: 142 --- Code Execution Successful ---
---	--

4. 0/1 Knapsack Problem

Program

<pre>main.py 1 def knapsack(wt, val, W): 2 n = len(wt) 3 dp = [[0] * (W + 1) for _ in range(n + 1)] 4 5 for i in range(1, n + 1): 6 for w in range(W + 1): 7 if wt[i - 1] <= w: 8 dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w]) 9 else: 10 dp[i][w] = dp[i - 1][w] 11 return dp[n][W] 12 13 wt = [1, 3, 4, 5] 14 val = [1, 4, 5, 7] 15 W = 7 16 print("Maximum Profit =", knapsack(wt, val, W))</pre>	Output Maximum Profit = 9 --- Code Execution Successful ---
--	---

5. 0/1 Knapsack Using Memoization

Program

main.py	Run	Output
<pre>1 def knapsack(W, wt, val, n, memo): 2 if n == 0 or W == 0: 3 return 0 4 if memo[n][W] != -1: 5 return memo[n][W] 6 7 if wt[n - 1] <= W: 8 memo[n][W] = max(9 val[n - 1] + knapsack(W - wt[n - 1], wt, val, n - 1, memo), 10 knapsack(W, wt, val, n - 1, memo) 11) 12 else: 13 memo[n][W] = knapsack(W, wt, val, n - 1, memo) 14 return memo[n][W] 15 16 wt = [1, 3, 4, 5] 17 val = [1, 4, 5, 7] 18 W = 7 19 memo = [[-1] * (W + 1) for _ in range(len(wt) + 1)] 20 print("Maximum Profit =", knapsack(W, wt, val, len(wt), memo))</pre>		Maximum Profit = 9 --- Code Execution Successful ---

6. Subset Sum Problem

Program

main.py	Run	Output
<pre>1 def subset_sum(arr, target): 2 n = len(arr) 3 dp = [[False] * (target + 1) for _ in range(n + 1)] 4 5 for i in range(n + 1): 6 dp[i][0] = True 7 8 for i in range(1, n + 1): 9 for j in range(1, target + 1): 10 if arr[i - 1] <= j: 11 dp[i][j] = dp[i - 1][j] or dp[i - 1][j - arr[i - 1]] 12 else: 13 dp[i][j] = dp[i - 1][j] 14 return dp[n][target] 15 16 print(subset_sum([3, 34, 4, 12, 5, 2], 9))</pre>		True --- Code Execution Successful ---

7. Travelling Salesperson Problem (DP Version)

Program

<pre>main.py 1 from functools import lru_cache 2 3 graph = [4 [0, 10, 15, 20], 5 [10, 0, 35, 25], 6 [15, 35, 0, 30], 7 [20, 25, 30, 0] 8] 9 n = len(graph) 10 11 @lru_cache(None) 12 def tsp(mask, pos): 13 if mask == (1 << n) - 1: 14 return graph[pos][0] 15 ans = float('inf') 16 for city in range(n): 17 if not mask & (1 << city): 18 ans = min(ans, graph[pos][city] + tsp(mask (1 << city), city)) 19 return ans 20 21 print("Minimum Cost =", tsp(1, 0))</pre>	Output Minimum Cost = 80 --- Code Execution Successful ---
--	--

8. Travelling Salesperson Using DP Table

Program

<pre>main.py 1 def tsp_dp(graph): 2 n = len(graph) 3 dp = [[float('inf')] * n for _ in range(1 << n)] 4 dp[1][0] = 0 5 6 for mask in range(1 << n): 7 for u in range(n): 8 if mask & (1 << u): 9 for v in range(n): 10 if not mask & (1 << v): 11 dp[mask (1 << v)][v] = min(12 dp[mask (1 << v)][v], 13 dp[mask][u] + graph[u][v] 14) 15 16 ans = float('inf') 17 for i in range(1, n): 18 ans = min(ans, dp[(1 << n) - 1][i] + graph[i][0]) 19 return ans 20 21 graph = [22 [0, 10, 15, 20], 23 [10, 0, 35, 25], 24 [15, 35, 0, 30], 25 [20, 25, 30, 0] 26] 27 print("Minimum Cost =", tsp_dp(graph))</pre>	Output Minimum Cost = 80 --- Code Execution Successful ---
---	--

9. Optimal Binary Search Tree (OBST)

Program

<pre>main.py 1 def optimal_bst(freq): 2 n = len(freq) 3 cost = [[0] * n for _ in range(n)] 4 5 for i in range(n): 6 cost[i][i] = freq[i] 7 8 for L in range(2, n + 1): 9 for i in range(n - L + 1): 10 j = i + L - 1 11 cost[i][j] = float('inf') 12 total = sum(freq[i:j + 1]) 13 for r in range(i, j + 1): 14 left = cost[i][r - 1] if r > i else 0 15 right = cost[r + 1][j] if r < j else 0 16 cost[i][j] = min(cost[i][j], left + right + total) 17 return cost[0][n - 1] 18 19 print("Optimal Cost =", optimal_bst([34, 8, 50]))</pre>	Output Optimal Cost = 142 --- Code Execution Successful ---
--	---

10. Matrix Chain Multiplication

Program

<pre>main.py 1 def matrix_chain(arr): 2 n = len(arr) 3 dp = [[0] * n for _ in range(n)] 4 5 for L in range(2, n): 6 for i in range(1, n - L + 1): 7 j = i + L - 1 8 dp[i][j] = float('inf') 9 for k in range(i, j): 10 q = dp[i][k] + dp[k + 1][j] + arr[i - 1] * arr[k] * arr[j] 11 dp[i][j] = min(dp[i][j], q) 12 return dp[1][n - 1] 13 14 print("Minimum Multiplications =", matrix_chain([10, 20, 30, 40, 30]))</pre>	Output Minimum Multiplications = 30000 --- Code Execution Successful ---
--	--

11. Coin Change (Minimum Coins)

Program

main.py		Run
<pre>1 def min_coins(coins, amount): 2 dp = [float('inf')] * (amount + 1) 3 dp[0] = 0 4 5 for i in range(1, amount + 1): 6 for c in coins: 7 if c <= i: 8 dp[i] = min(dp[i], dp[i - c] + 1) 9 return dp[amount] 10 11 print("Minimum Coins =", min_coins([1, 2, 5], 11))</pre>		Output Minimum Coins = 3 --- Code Execution Successful ---

12. Rod Cutting Problem

Program

main.py		Run
<pre>1 def rod_cut(price, n): 2 dp = [0] * (n + 1) 3 4 for i in range(1, n + 1): 5 for j in range(i): 6 dp[i] = max(dp[i], price[j] + dp[i - j - 1]) 7 return dp[n] 8 9 print("Maximum Revenue =", rod_cut([1, 5, 8, 9, 10, 17, 17, 20], 8))</pre>		Output Maximum Revenue = 22 --- Code Execution Successful ---

13. Minimum Cost Path

Program

main.py

Run

```
1 def min_cost(cost):
2     m = len(cost)
3     n = len(cost[0])
4     dp = [[0] * n for _ in range(m)]
5     dp[0][0] = cost[0][0]
6
7     for i in range(1, m):
8         dp[i][0] = dp[i - 1][0] + cost[i][0]
9     for j in range(1, n):
10        dp[0][j] = dp[0][j - 1] + cost[0][j]
11
12    for i in range(1, m):
13        for j in range(1, n):
14            dp[i][j] = cost[i][j] + min(dp[i - 1][j], dp[i][j - 1])
15    return dp[m - 1][n - 1]
16
17 cost = [
18     [1, 3, 1],
19     [1, 5, 1],
20     [4, 2, 1]
21 ]
22 print("Minimum Cost =", min_cost(cost))
```

Output

Minimum Cost = 7

--- Code Execution Successful ---